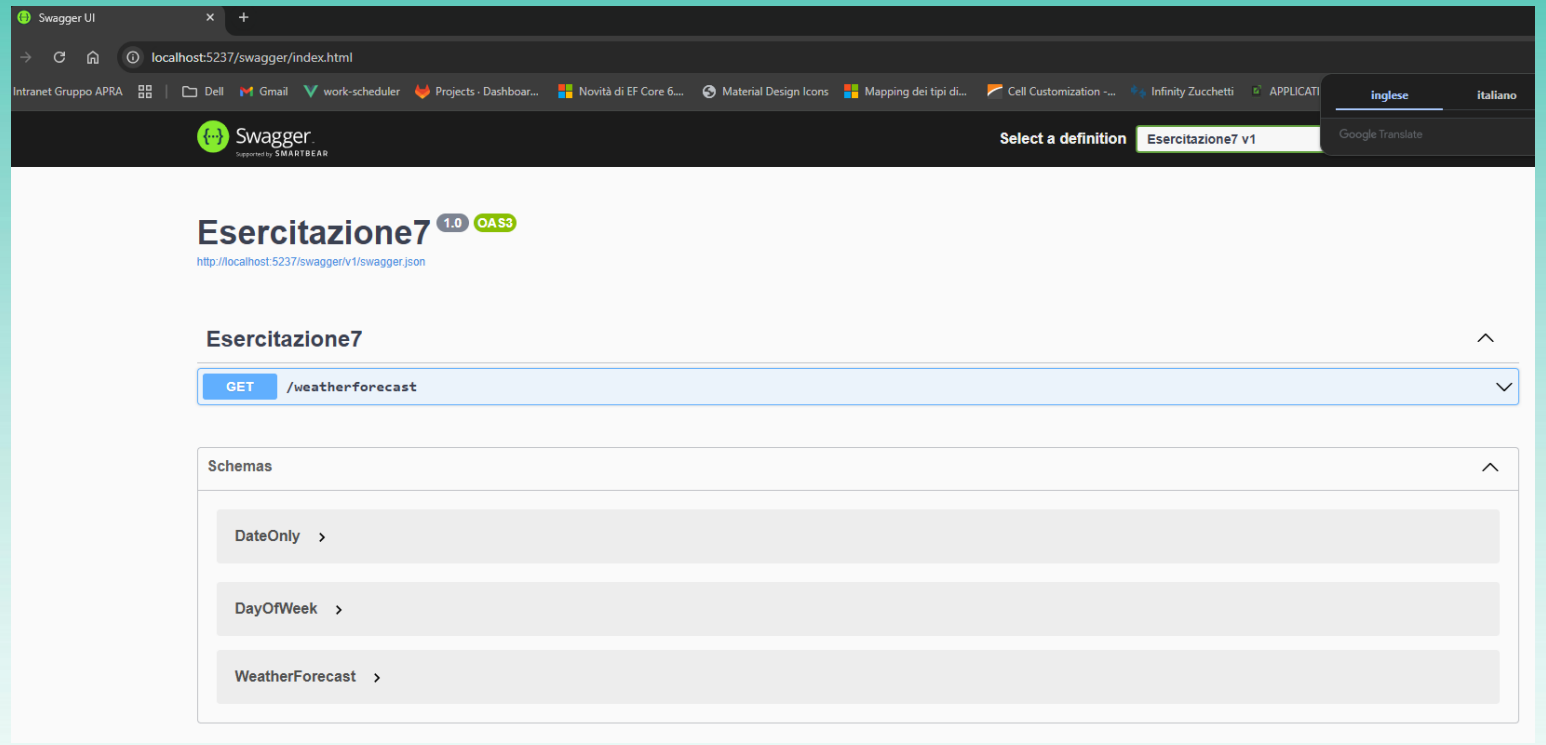
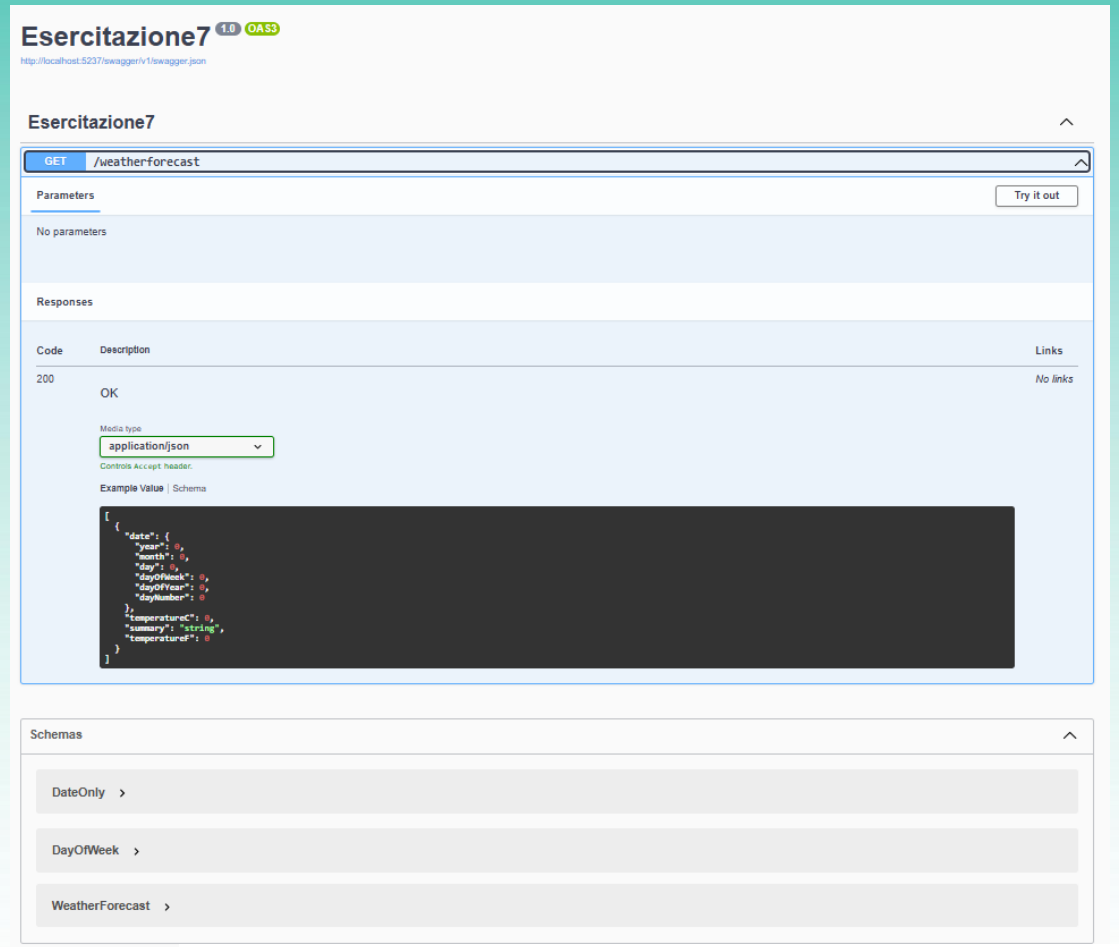
Esercitazione7 – Web API



Quella che appare è l'interfaccia grafica di swagger, da cui possiamo testare le api del progetto.

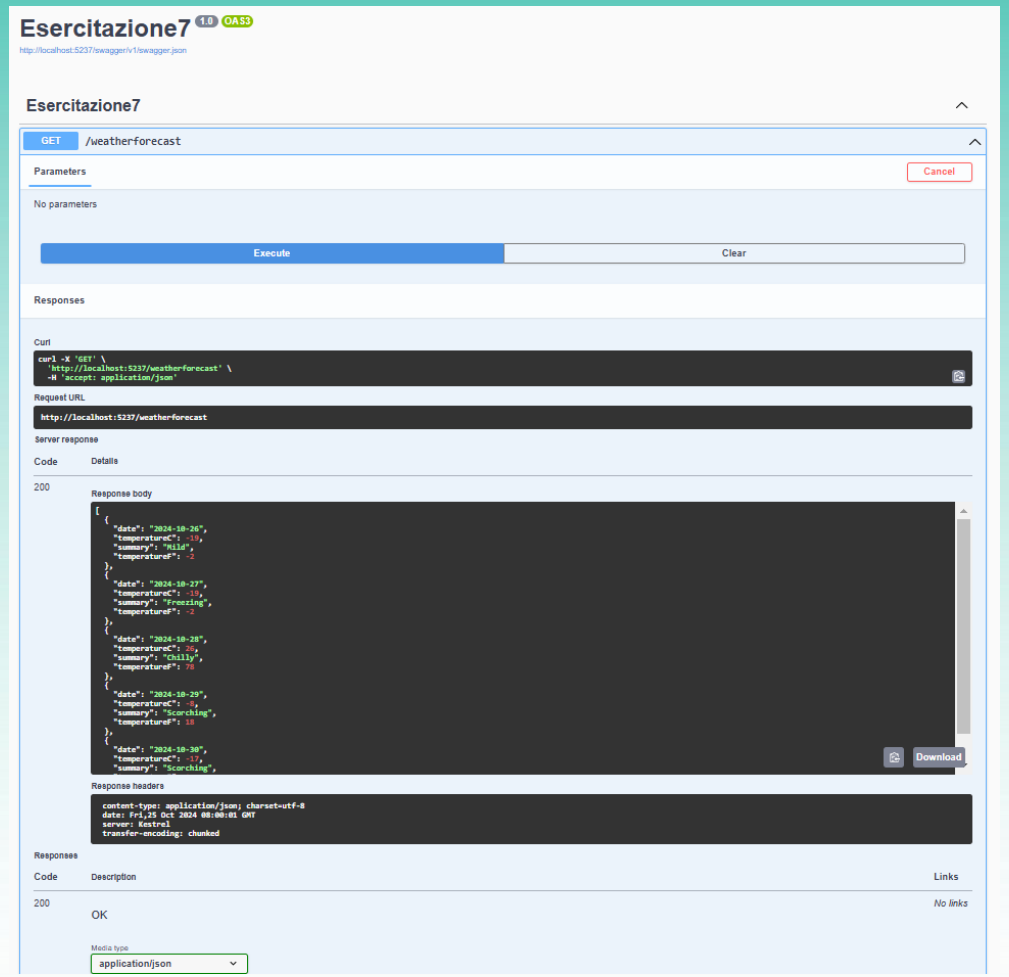
Esercitazione7 – Web API



Vediamo:

1. Il nome del progetto (Esercitazione7);
2. La chiamata HTTP GET e l'endpoint (/weatherforecast);
3. Parameters, se presenti (in questo caso No parameters);
4. Responses, sono le risposte accettate, in questo caso:
   - Status CODE: 200
   - Tipo della risposta: application/json.

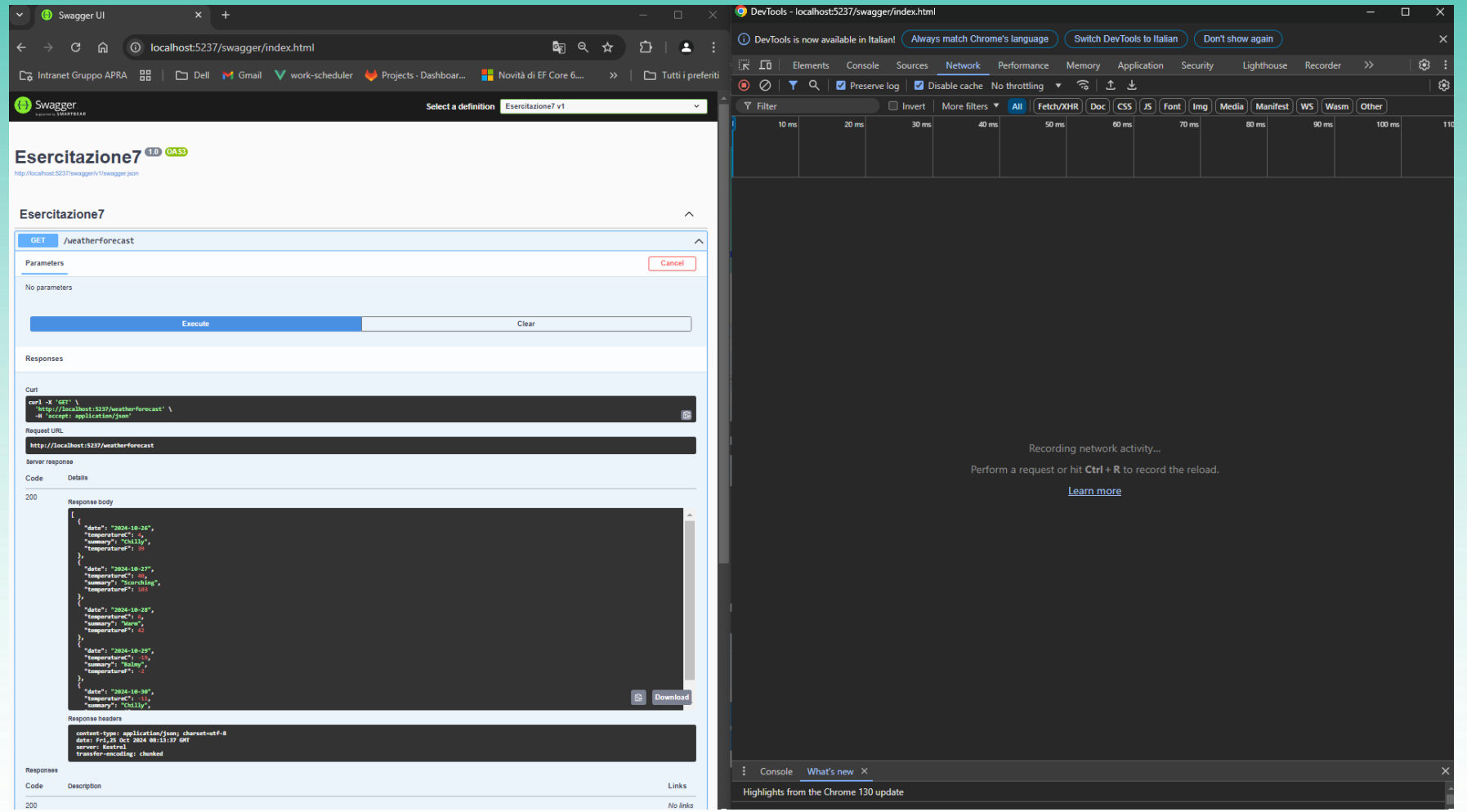
Esercitazione7 – Web API



Per eseguire, cliccare su Try it out e Execute.
Otteniamo la risposta (Response body).

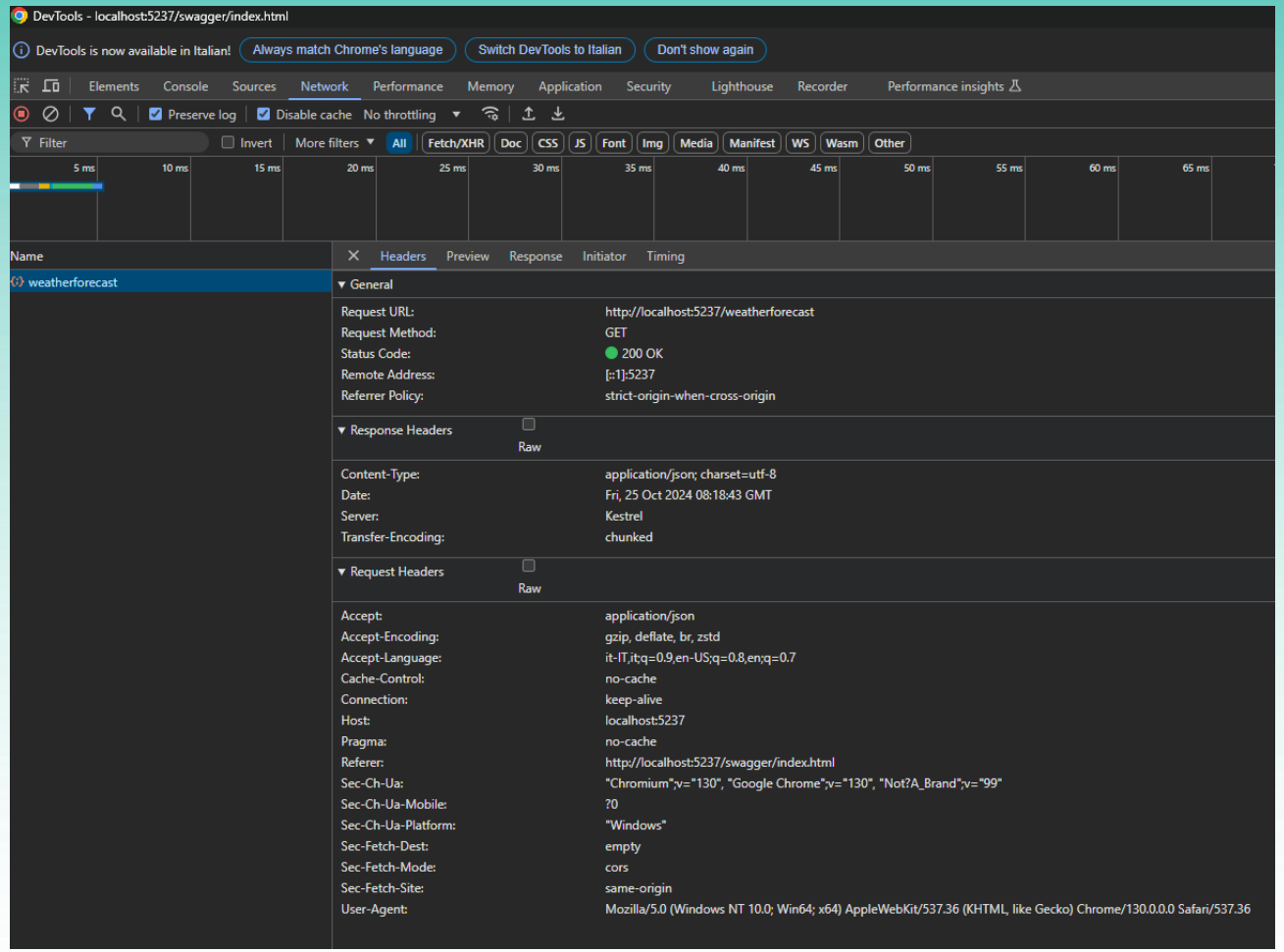
Ora clicchiamo F12 per aprire gli strumenti dello sviluppatore e posizioniamoci nella tab Network, come nella slide successiva

Esercitazione7 – Web API



Se da qui ora clicchiamo su Execute, vediamo che la tab Network si popola con una riga, se clicchiamo in quella riga vediamo tutte le informazioni relative alla chiamata HTTP, come nella slide successiva

Esercitazione7 – Web API



Nella tab:

- Headers, ritroviamo le informazioni generali, con i dettagli della chiamata: l'URL(Request URL), il tipo di chiamata(Request Method), lo status code della risposta e il tipo di risposta accettata(Accept);
- Preview o Response, ritroviamo i dati inviati dal server come risposta, se presenti;
- Se avessimo avuto dei parametri nel body avremmo visualizzato anche una ulteriore tab (Payload) con i dati inviati dal client al server.

Esercitazione7.1 – Web API (.NET 10)

- Creare un nuovo progetto, come prima ma su .NET 10
  - `dotnet new webapi -n 7.1-DemoApi10 -f net10.0`
- Ed eseguire quanto fatto in precedenza per consumare API
  - Che cosa succede?

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

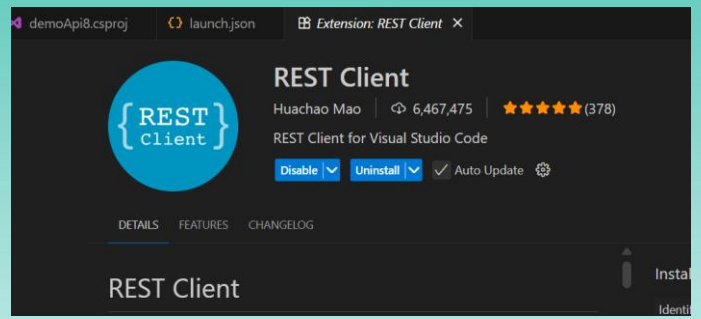
Esercitazione7 – Web API (.NET 10)

- Creare un nuovo progetto, come prima ma su .NET 10
  - `dotnet new webapi -n 7demoApi10 -f net10.0`
- Ed eseguire quanto fatto in precedenza per consumare API
  - Che cosa succede? https://medium.com/@dev.mjdhanesh/scalar-api-reference-in-net-9-a-modern-alternative-to-swagger-cf2d48cdd258
  - Aggiungere e configurare Scalar
- Utilizzare un client REST esterno:
  - REST client (VS code ext)
  - Postman,
  - Bruno

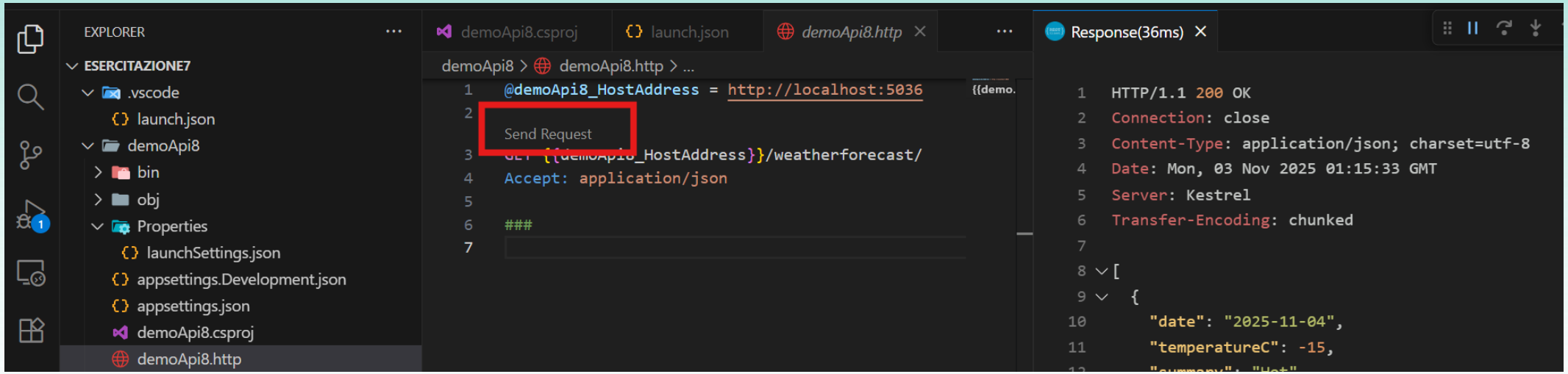




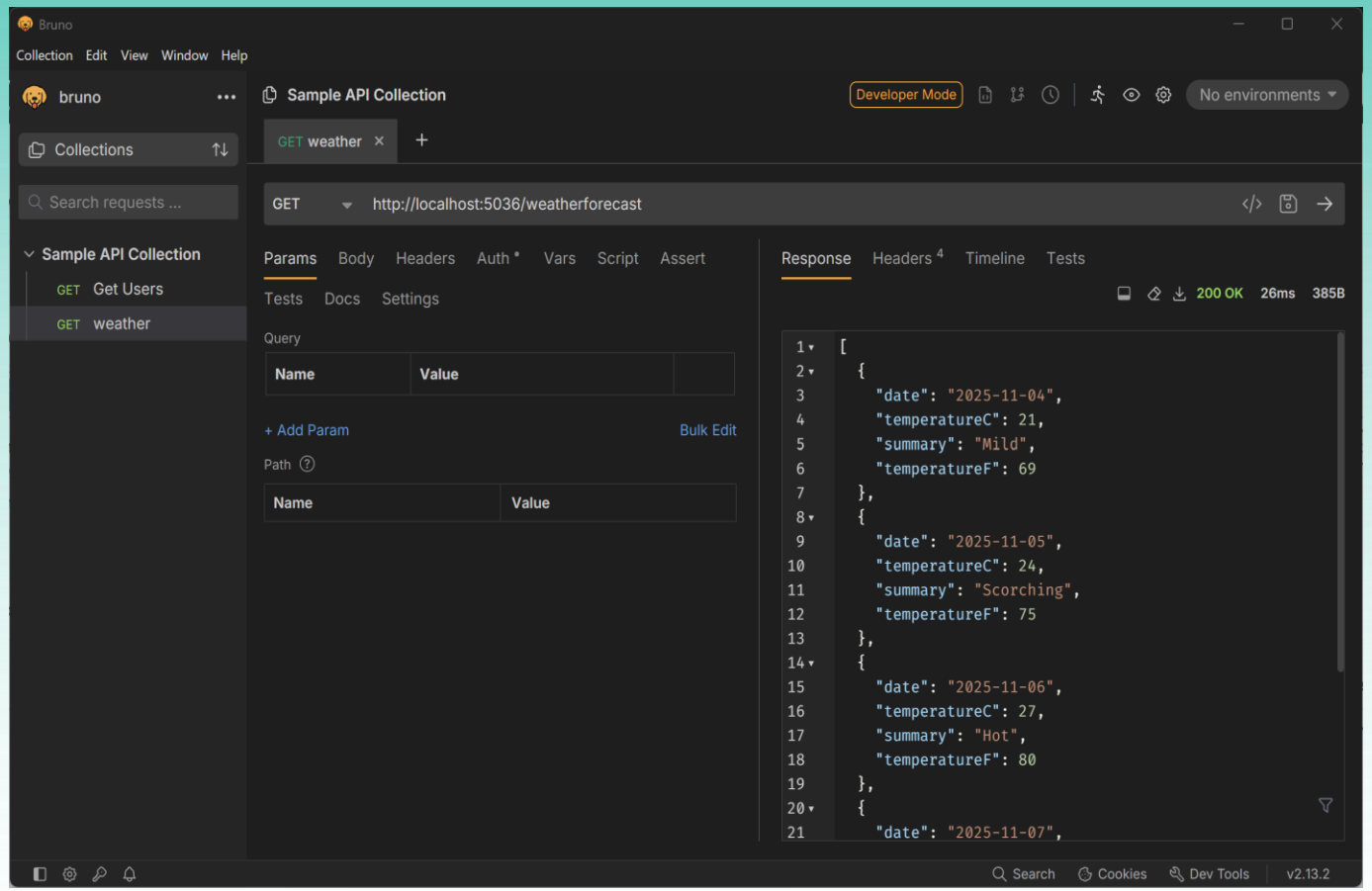
Esercitazione7 – Web API (REST Client)



- Ha come vantaggio la possibilità di sfruttare al volo le richieste .http nella codebase
- Come testare?



Esercitazione7 – Web API (Bruno)



Perché Bruno e non Postman?

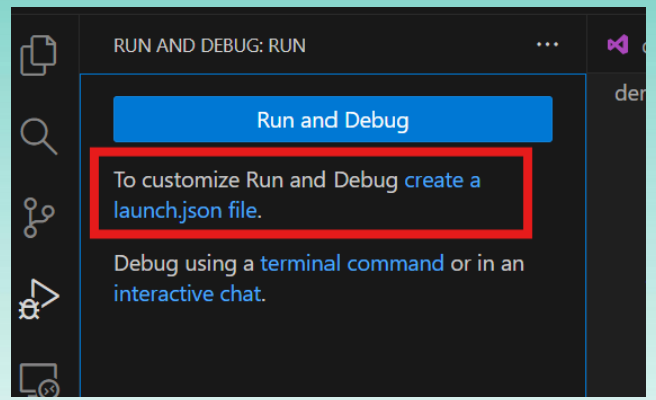
Bruno

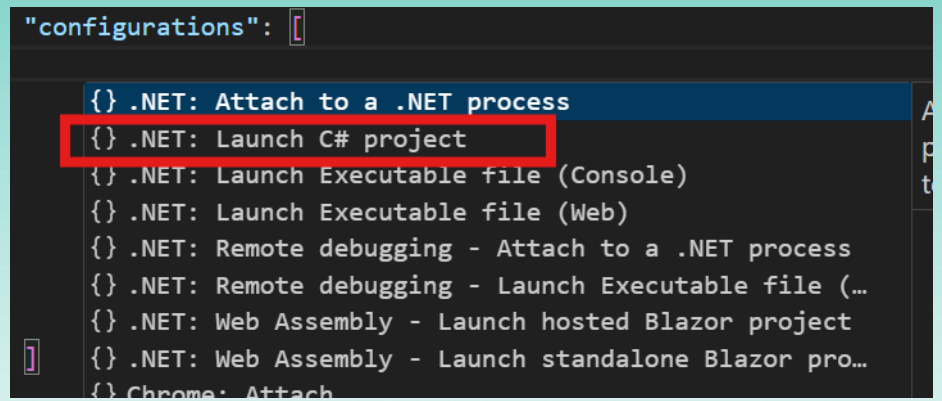
https://www.usebruno.com/compare/bruno-vs-postman

Bruno semplifica il test delle API grazie a un'interfaccia grafica, gestione di collezioni e variabili, e strumenti avanzati che migliorano la produttività rispetto a client REST più semplici.

Esercitazione7 – Launcher VS Code

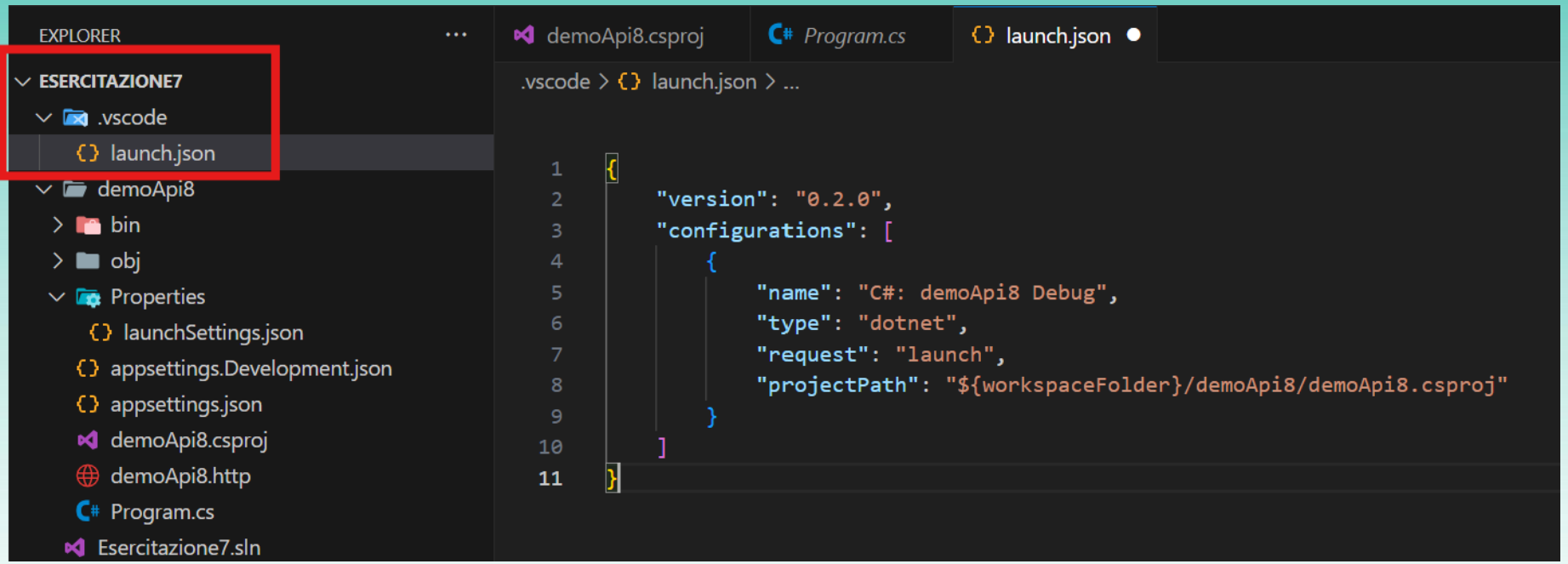
- Creare una configurazione per il debug di VS code (tipo debugger, progetto di avvio, gli argomenti, variabili di ambiente)
- Selezionare C#






```
// For more information, visit: https://go.microsoft.com/fwlink/?linkid=830387
"version": "0.2.0",
"configurations": [
    {
        "name": "C#: <project-name> Debug",
        "type": "dotnet",
        "request": "launch",
        "projectPath": "${workspaceFolder}/<relative-path-to-project-folder><project-name>.csproj"
    }
```

Esercitazione7 – Launcher VS Code



Provare da scorciatoia F5 (o FN+F5 a seconda dell'impostazione del bios)

Esercitazione7 – Launcher VS Code

E per lanciare entrambi i progetti insieme?

Esercitazione7 – Launcher VS Code

….provatelo a casa!!!

Suggerimento 👇
https://learn.microsoft.com/en-us/microsoft-edge/visual-studio-code/microsoft-edge-devtools-extension/launch-json

Esercitazione8 – Web API Person

- Nel terminale di Visual Studio Code spostarsi nella directory in cui si vuole creare il nuovo progetto;
- Creare un nuovo progetto .NET 8 di tipo web API denominato Esercitazione8
  - `dotnet new webapi -n Esercitazione8`

Esercitazione8 – Web API Person

```
public class Person
{
    public int id { get; set; }
    public string? Name { get; set; }
    public int Age { get; set; }
}
```

Person.cs
Creiamo una classe Person come in figura

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione8 – Web API Person

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
```

Program.cs
Personalizziamo ora il nostro Program, sempre utilizzando l'interfaccia grafica di Swagger, ma utilizzando le nostre API.

A fianco vediamo la prima parte del codice, essenzialmente uguale al Program.cs che avevamo nella precedente Esercitazione.

Esercitazione8 – Web API Person

```
Dictionary<int, Person> dict = new Dictionary<int, Person>();
app.MapGet("/getPeople", () =>
{
    if(dict.Count > 0)
        return dict.Values.ToArray();
    else
        return [];
});
app.MapGet("/getPerson/{id}", (int id) =>
{
    if(dict.Count > 0 && dict.ContainsKey(id))
        return dict[id];
    else
        return null;
});
```

Program.cs

Utilizziamo un Dictionary di tipo chiave-valore per simulare una tabella del DB (la chiave corrisponde all'id della persona, i valori a Person), ogni elemento di dict corrisponde a una persona con chiave univoca id.

Inseriamo poi le API:

- getPeople chiamata GET senza parametri che controlla se dict contiene degli elementi, in tal caso restituisce un array di tutte i valori di dict (tutte le Person sotto forma di array), altrimenti restituisce un array vuoto;
- getPerson/id chiamata GET con parametro id (nell'url) che controlla che dict contiene degli elementi e che contiene la chiave id, in tal caso restituisce un valore Person di dict con chiave uguale a id (dict[id]), altrimenti restituisce un array null;

Esercitazione8 – Web API Person

```
app.MapPost("/create", (Person person) =>
{
    if (dict.Count > 0 && dict.ContainsKey(person.id))
        return Results.BadRequest();
    else
    {
        dict.Add(person.id, person);
        return Results.Ok();
    }
});
app.MapPut("/update", (Person person) =>
{
    if ((dict.Count > 0 && !dict.ContainsKey(person.id)) || dict.Count == 0)
        return Results.NotFound();
    else
    {
        dict[person.id] = person;
        return Results.Ok();
    }
});
```

Program.cs

- Create chiamata POST con parametro di tipo Person (nel body) che controlla se dict contiene elementi e che l'id del parametro person è presente come chiave in dict, in tal caso restituisce BadRequest, altrimenti inserisce un nuovo elemento in dict (con chiave person.id e valore person) e restituisce OK.
- Update chiamata PUT con parametro di tipo Person (nel body) che controlla se dict contiene elementi e che l'id di person non è presente come chiave in dict oppure controlla che dict non contiene elementi, in entrambi i casi restituisce NotFound (quindi è sufficiente che uno dei due casi si verifichi), altrimenti modifica l'elemento di dict con chiave uguale a person.id con i nuovi valori del parametro person e restituisce OK;

Esercitazione8 – Web API Person

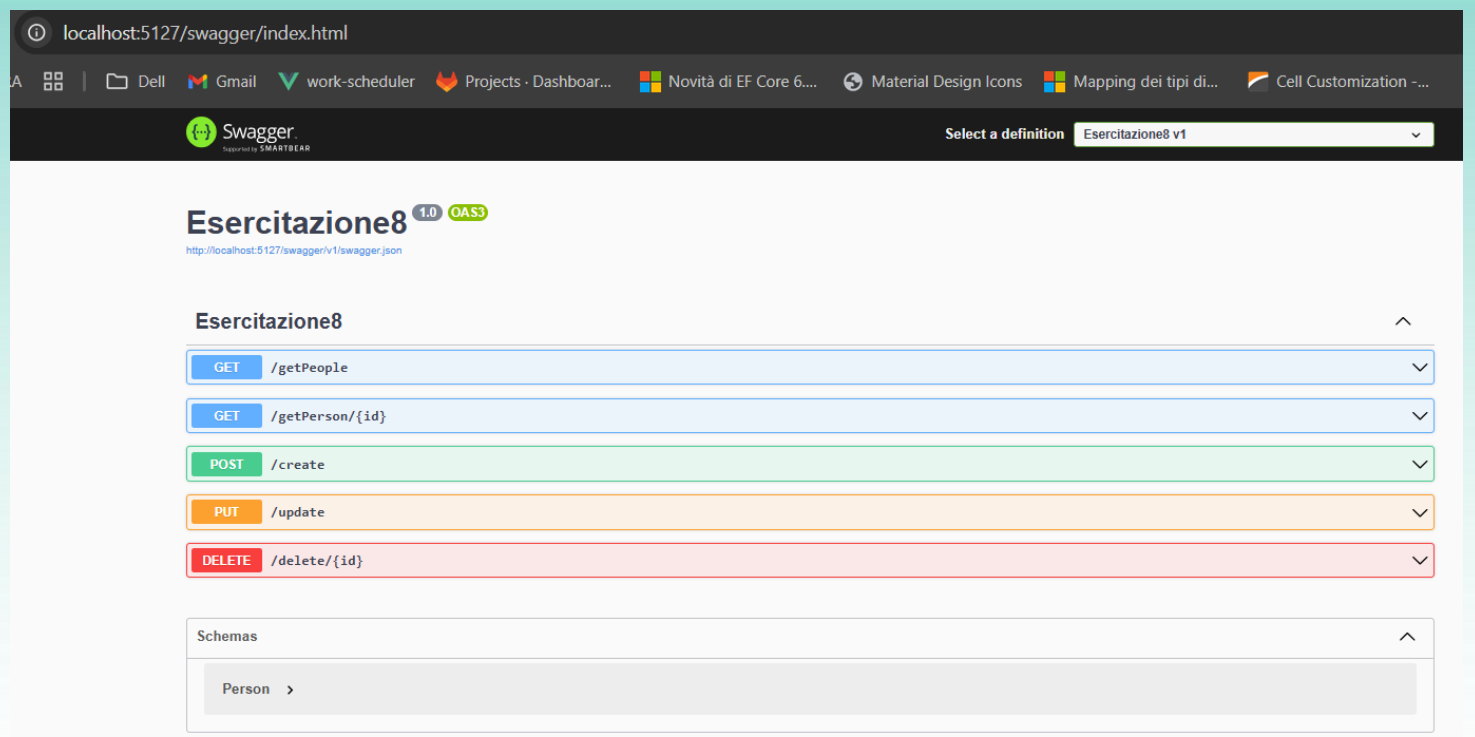
```
app.MapDelete("/delete/{id}", (int id) =>
{
    if ((dict.Count > 0 && !dict.ContainsKey(id)) ||
dict.Count == 0)
        return Results.NotFound();
    else
    {
        dict.Remove(id);
        return Results.Ok();
    }
});
app.Run();
```

Program.cs

- Delete/id chiamata DELETE con parametro id (nell'url) che controlla se dict contiene elementi e che l'id non è presente come chiave in dict oppure che dict non contiene elementi, in entrambi i casi restituisce NotFound (quindi è sufficiente che uno dei due casi si verifichi), altrimenti elimina l'elemento di dict con chiave id e restituisce OK.

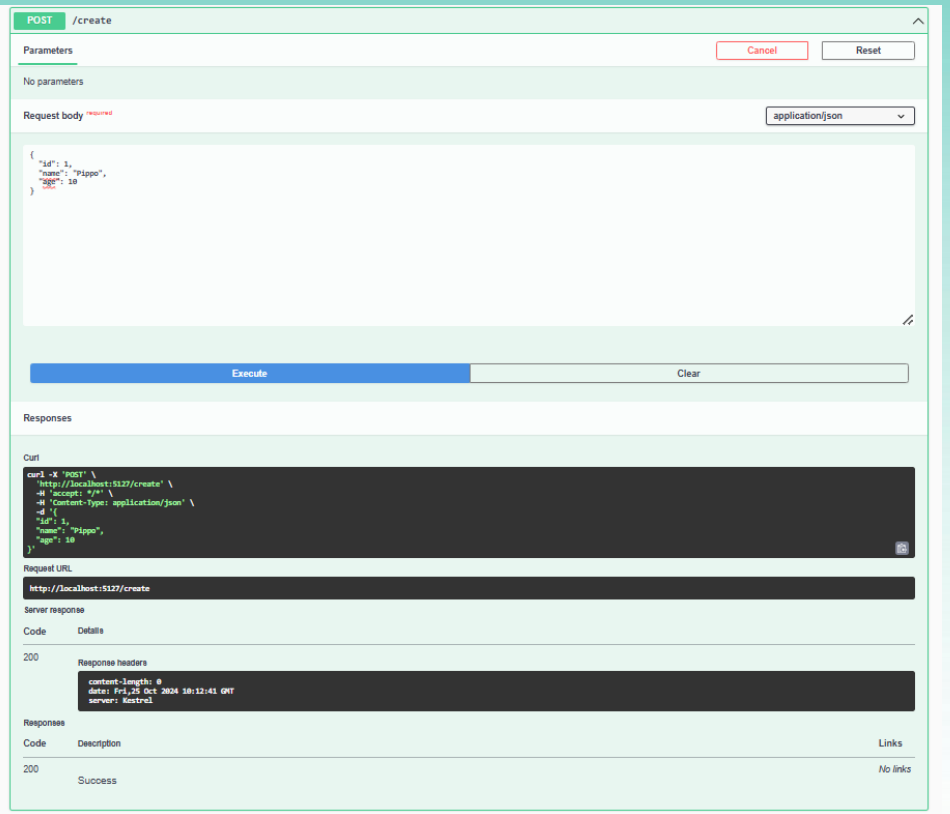
Esercitazione8 – Web API Person

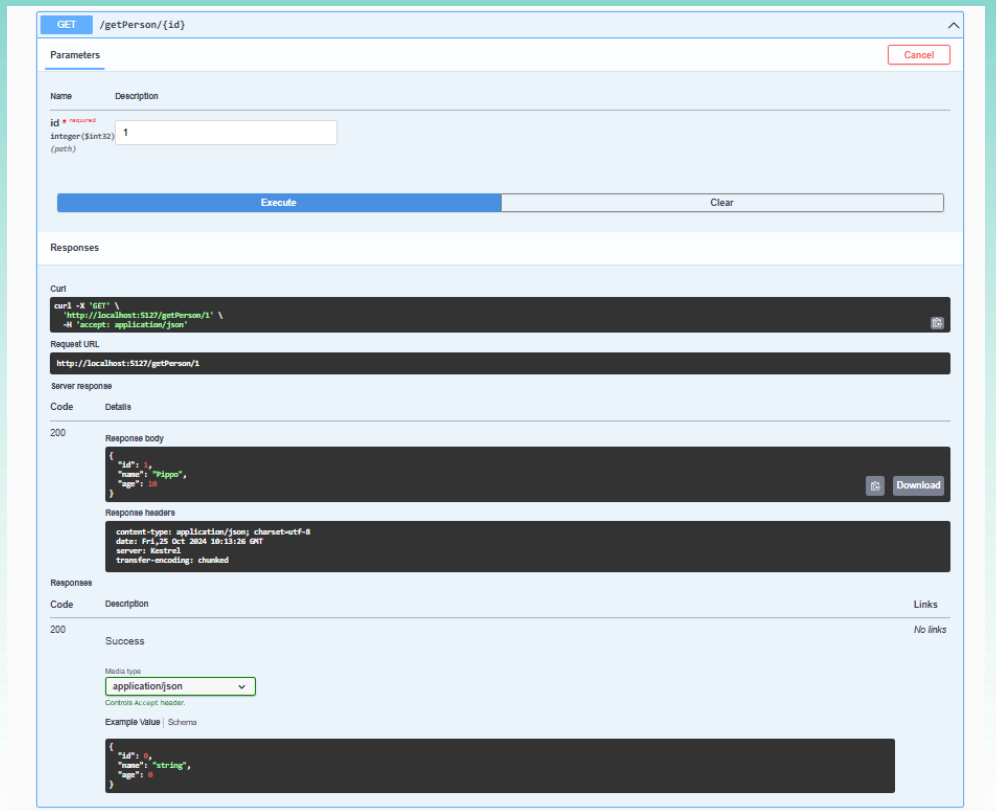
Avviando l'app da terminale con il comando
dotnet run
e aprendo Swagger nel browser possiamo testare le api:



Esercitazione8 – Web API Person

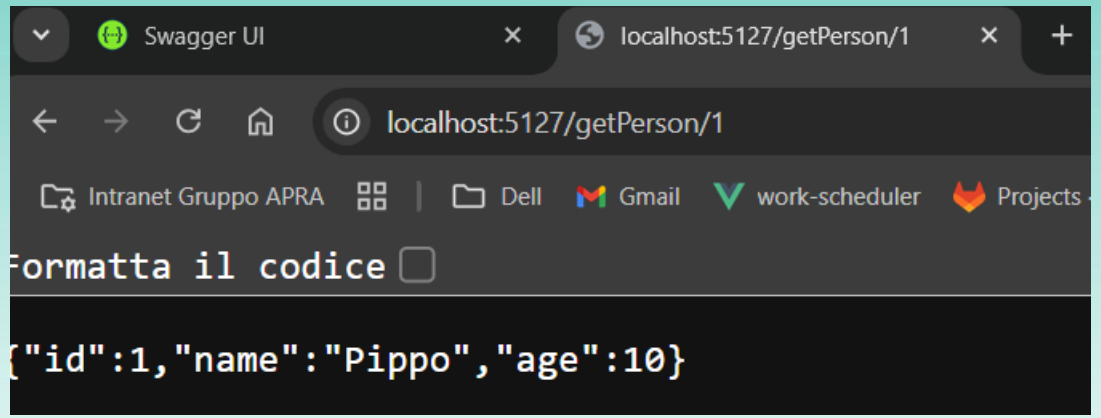
Esempio di inserimento di una persona e successiva lettura tramite id (notare che la POST passa un json come parametro, la GET passa il parametro nell'url):





Esercitazione8 – Web API Person

E’ possibile fare la stessa chiamata precedente anche in una nuova scheda del browser come in seguito:



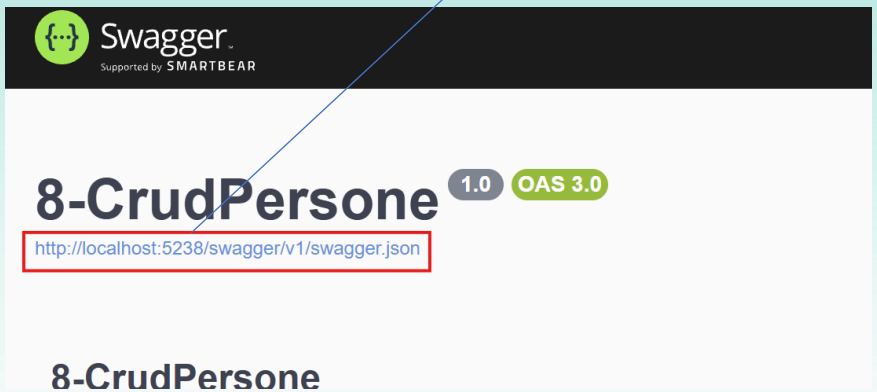
Quindi non ci stiamo interfacciando con Swagger, ma stiamo richiamando direttamente l’endpoint getPerson con parametro id = 1

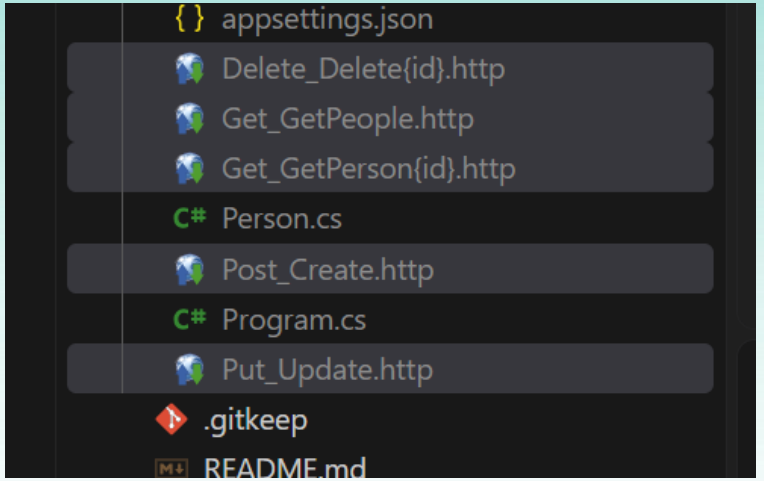
Esercitazione8 – Web API Person

Generare i file .http in automatico

Non serve farli a mano, ma basta generarli dalla descrizione OpenAPI (Swagger/Scalar) dell'API.

- Installa lo strumento (una sola volta):
  dotnet tool install --global httpgenerator
- Avvia l'API
- Lancia il comando:
  httpgenerator <link-openApi.json> --output-type [OneRequestPerFile][OneFile] --output





Esercitazione8.1 – Web API Piloti

ORA TOCCA A VOI!

- Un'API CRUD come quella delle persone, ma per i piloti di F1.
- Modello da eseguire: `\Exercises\8.1-CrudPiloti`
- Consegna completa nel README
- Si lavora in `\MyExercises\8.1-CrudPiloti`



COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione9 – CSR

Cerchiamo di dare un’architettura pulita alla nostra applicazione.

Quindi partendo dall’Esercitazione8, la nuova applicazione avrà le stesse funzionalità precedenti, ma la struttura del progetto verrà modificata in modo da rispettare la CLEAN ARCHITECTURE.

Oggi nel mondo del software è molto diffuso l’uso delle cosiddette Clean Architecture. Vengono chiamate così perché si basano tutte sullo stesso principio di progettazione del software: la separazione delle responsabilità.

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Esercitazione9 – CSR

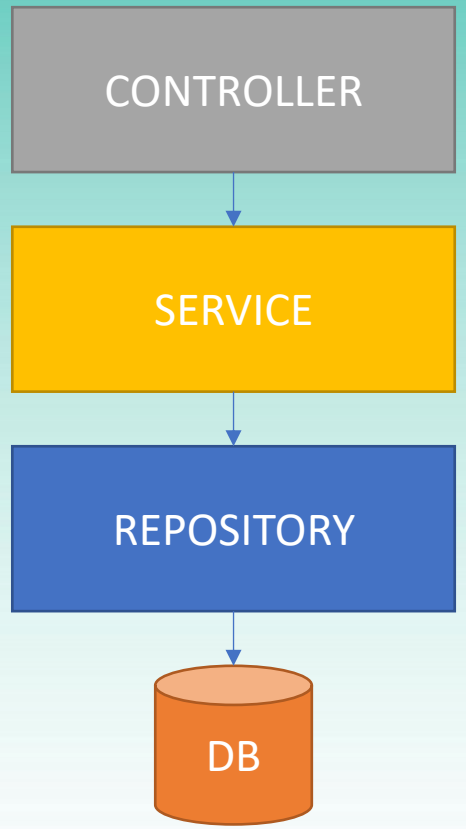
MODELLO CSR

Nello sviluppo software, creare applicazioni scalabili, manutenibili e testabili è fondamentale.

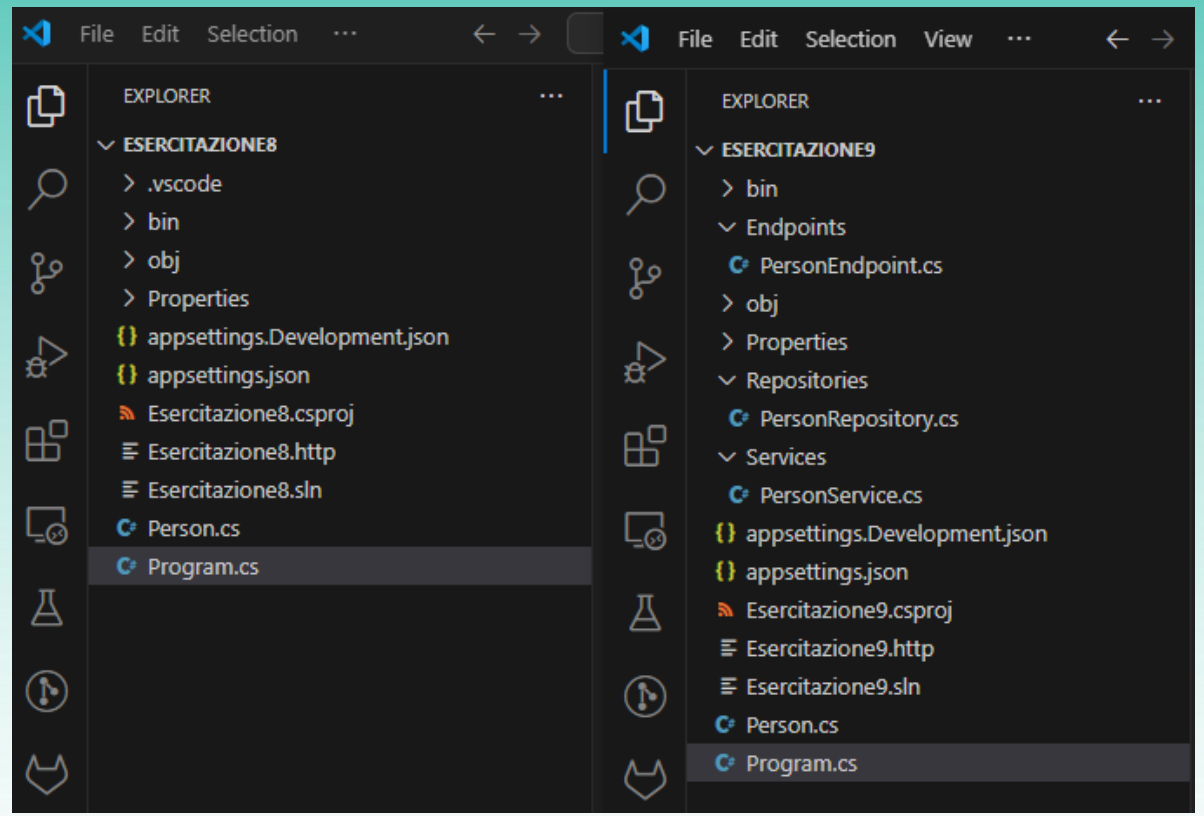
Un modello architettonico che aiuta a raggiungere questi obiettivi è il modello Controller-Service-Repository (CSR). Questo modello promuove la separazione delle preoccupazioni dividendo la logica dell'applicazione in tre livelli distinti: il Controller, il Service e il Repository.

Ogni livello ha una responsabilità specifica, rendendo il codice più facile da gestire, testare e scalare:

- il livello Controller gestisce le richieste HTTP in arrivo, le elabora e restituisce le risposte HTTP appropriate. Agisce come intermediario tra il client e il livello Service;
- il livello Service contiene la logica aziendale dell'applicazione. Elabora i dati ricevuti dal controller, esegue le operazioni necessarie e comunica con il livello di Repository per recuperare o rendere persistenti i dati;
- il livello Repository, gestisce l'accesso ai dati e la persistenza. Comunica con il database o altre fonti di dati, incapsulando la logica per interrogare, salvare e aggiornare i dati.



Esercitazione9 – CSR



Differenze fra Esercitazione8 e Esercitazione9

Mettiamo a confronto l'architettura delle due applicazioni:

- Nell'Esercitazione8 avevamo creato una solo classe Person all'interno del file Person.cs e endpoint, logica dell'applicazione e accesso ai dati avvenivano all'interno del Program.cs;
- Nell'Esercitazione9 sono invece presenti altre cartelle che permettono di rispettare la separazione delle responsabilità:
    1. Il livello Controller lo troviamo all'interno del file PersonEndpoint.cs della cartella Endpoints;
    2. Il livello Service lo troviamo all'interno del file PersonService.cs della cartella Services;
    3. Il livello Repository lo troviamo all'interno del file PersonRepository.cs della cartella Repositories.

Le cartelle Endpoints, Services e Repositories ci permettono di suddividere le responsabilità e di farlo per ogni singolo modello (classe).
Quindi, se oltre alla classe Person di Person.cs in futuro aggiungeremo un'altra classe (ad esempio la classe Animal in un file Animal.cs), aggiungeremo altri file nelle cartelle Endpoints, Services e Repositories: esattamente un ulteriore file per ogni livello di responsabilità, cioè per ogni cartella (rispettivamente i files AnimalEndpoint.cs, AnimalService.cs e AnimalRepository.cs).

Esercitazione9 – CSR

```
var builder = WebApplication.CreateBuilder(args);

// Add services to the container.

// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle

builder.Services.AddEndpointsApiExplorer();

builder.Services.AddSwaggerGen();

var app = builder.Build();

app.MapAPIEndpoints();

// Configure the HTTP request pipeline.

if (app.Environment.IsDevelopment())

{

    app.UseSwagger();

    app.UseSwaggerUI();

}

app.UseHttpsRedirection();

app.Run();
```

Program.cs

Rispetto all'Esercitazione8, nel Program.cs, togliamo tutto il codice che appartiene ai livelli di responsabilità CSR, per spostarli in altri files.
Ricordiamo però che il Program.cs è il punto di avvio dell'applicazione ed è quello che si occupa di creare ed eseguire l'app.
E' quindi necessario dire all'app di aggiungere le nostre API, altrimenti non verrano create.
Per farlo richiamiamo un metodo, statico, creato da noi: MapAPIEndpoints() che contiene la definizione delle API.
Vediamo con la slide successiva la definizione del metodo MapAPIEndpoints() della classe PersonEndpoints.

Esercitazione9 – CSR

```
public static class PersonEndpoints

{

    private static PersonService personService = new
PersonService();

    public static void MapAPIEndpoints(this
IEndpointRouteBuilder app)

    {

        app.MapGet("/getPeople", () =>
personService.GetPeople());

        app.MapGet("/getPerson/{id}", (int id) =>
personService.GetPerson(id));

        app.MapPost("/create", (Person person) =>

        {

            bool result =
personService.Create(person);

            if (result == true)

                return Results.Ok();

            else

                return Results.BadRequest();

        });
```

```
        app.MapPut("/update", (Person person) =>

        {

            bool result =
personService.Update(person);

            if (result == true)

                return Results.Ok();

            else

                return Results.NotFound();

        });

        app.MapDelete("/delete/{id}", (int id) =>

        {

            bool result = personService.Delete(id);

            if (result == true)

                return Results.Ok();

            else

                return Results.NotFound();

        });

    }

}
```

PersonEndpoints.cs
Il file contiene la definizione di una classe statica: PersonEndpoints.

Questa classe è quella che deve contenere il livello Controller e quindi la definizione delle API, ma non deve contenere le responsabilità del livello Service e del livello Repository.

Ricordiamo che una classe statica non può essere istanziata e può contenere solo membri statici.

La classe PersonEndpoints ha:

- Un istanza statica di PersonService, che come vedremo è una classe che contiene il livello Service;
- Il metodo statico MapAPIEndpoints, che contiene la definizione delle API e le aggiunge all'app. Dentro le API non troviamo logica. Ogni API utilizza il membro statico personService della classe PersonEndpoints per richiamare i metodi della classe PersonService.

Ogni API della classe PersonEndpoints quindi richiama metodi del Service, prende la risposta e restituisce una risposta al chiamante (client).

Vediamo con la slide successiva la classe PersonService.

Esercitazione9 – CSR

```
public class PersonService

{

    private PersonRepository personRepository = new
PersonRepository();

    public IEnumerable<Person> GetPeople()

    {

        return personRepository.GetPeople();

    }

    public Person? GetPerson(int id)

    {

        return personRepository.GetPerson(id);

    }

    public bool Create(Person person)

    {

        return personRepository.Create(person);

    }

    public bool Update(Person person)

    {

        return personRepository.Update(person);

    }

    public bool Delete(int id)

    {

        return personRepository.Delete(id);

    }

}
```

PersonService.cs
Il file contiene la definizione di una classe: PersonService.

Questa classe è quella che deve contenere il livello Service e quindi la logica aziendale dell'applicativo, ma non deve contenere le responsabilità del livello Cotroller (già definito in PersonEndpoint) e del livello Repository.

La classe PersonService ha:

- Un istanza di PersonRepository, che come vedremo è una classe che contiene il livello Repository;
- La definizione dei metodi richiamati nelle API della classe PersonEndpoints. Questi metodi attualmente non contengono logica, nel nostro caso non stiamo manipolando i dati che ci arrivano in ritorno dal livello Repository, ma avremmo dovuto farlo qui, se necessario. Dentro i metodi non effettuiamo l'accesso diretto ai dati, perché ciò aspetta al livello Repository, infatti ogni metodo della classe PersonService utilizza il membro personRepository della classe PersonService per richiamare i metodi della classe PersonRepository.

Ogni metodo della classe PersonService quindi richiama metodi del Repository, prende la risposta e, dopo aver inserito la logica dell'applicativo, se presente, restituisce una risposta al chiamante (API del livello Controller).

Vediamo con la slide successiva la classe PersonRepository.

Esercitazione9 – CSR

```
public class PersonRepository

{

    private Dictionary<int, Person> dict = new Dictionary<int, Person>();

    public IEnumerable<Person> GetPeople()

    {

        if(dict.Count > 0)

            return dict.Values.ToArray();

        else

            return [];

    }

    public Person? GetPerson(int id)

    {

        if(dict.Count > 0 && dict.ContainsKey(id))

            return dict[id];

        else

            return null;

    }

    public bool Create(Person person)

    {

        if (dict.Count > 0 && dict.ContainsKey(person.id))

            return false;

        else

        {

            dict.Add(person.id, person);

            return true;

        }

    }
```

```
    public bool Update(Person person)

    {

        if ((dict.Count > 0 && !dict.ContainsKey(person.id)) || dict.Count == 0)

            return false;

        else

        {

            dict[person.id] = person;

            return true;

        }

    }

    public bool Delete(int id)

    {

        if ((dict.Count > 0 && !dict.ContainsKey(id)) || dict.Count == 0)

            return false;

        else

        {

            dict.Remove(id);

            return true;

        }

    }

}
```

PersonRepository.cs
Il file contiene la definizione di una classe: PersonRepository.

Questa classe è quella che deve contenere il livello Repository e quindi l'accesso ai dati dell'applicativo, ma non deve contenere le responsabilità del livello Cotroller (già definito in PersonEndpoint) e del livello Service (già definito in PersnService).

La classe PersonRepository ha:

- Un istanza di un Dictionary, che simula il nostro DB;
- La definizione dei metodi richiamati nel nella classe PersonService. Questi metodi si occupano di accedere, leggere e scrivere nei dati, che nel nostro caso sono rappresentati da dict. Ogni metodo della classe PersonRepository quindi prende i dati dal DB e li restituisce ai Controller, dopo aver inserito la logica dell'applicativo.

Ogni metodo della classe PersonRepository quindi si occupa di leggere e scrivere nei dati (nel nostro caso dict), e restituisce una risposta al chiamante (metodi del livello Service).

Risoluzione errori e launch progetti multipli

launch.json ● Program.cs

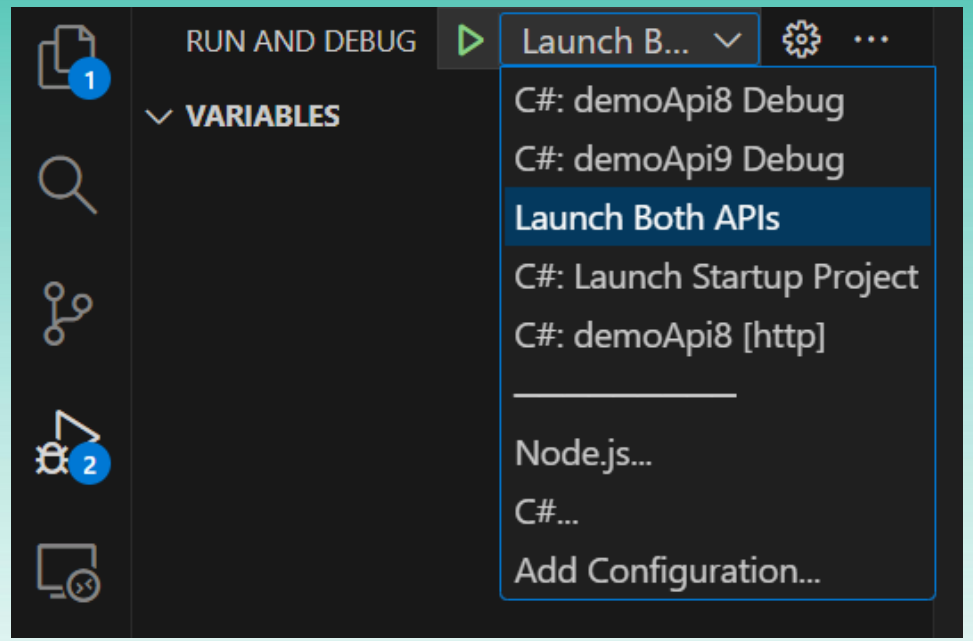
.vscode > launch.json > ...

```jsonc
{
    "version": "0.2.0",
    "configurations": [
        {
            "name": "C#: demoApi8 Debug",
            "type": "dotnet",
            "request": "launch",
            "projectPath": "${workspaceFolder}/demoApi8/demoApi8.csproj"
        },
        {
            "name": "C#: demoApi9 Debug",
            "type": "dotnet",
            "request": "launch",
            "projectPath": "${workspaceFolder}/demoApi9/demoApi9.csproj"
        }
    ],
    "compounds": [
        {
            "name": "Launch Both APIs",
            "configurations": [
                "C#: demoApi8 Debug",
                "C#: demoApi9 Debug"
            ],
            "stopAll": true
        }
    ]
}
```

https://learn.microsoft.com/en-us/microsoft-edge/visual-studio-code/microsoft-edge-devtools-extension/launch-json

- Aggiungere la configurazione del 2’ progetto all’array configurations[]
- Aggiungere sezione «compounds»: []
- stopAll: true, fermerà tutti i processi quando viene interrotto il debug

Risoluzione errori e launch progetti multipli

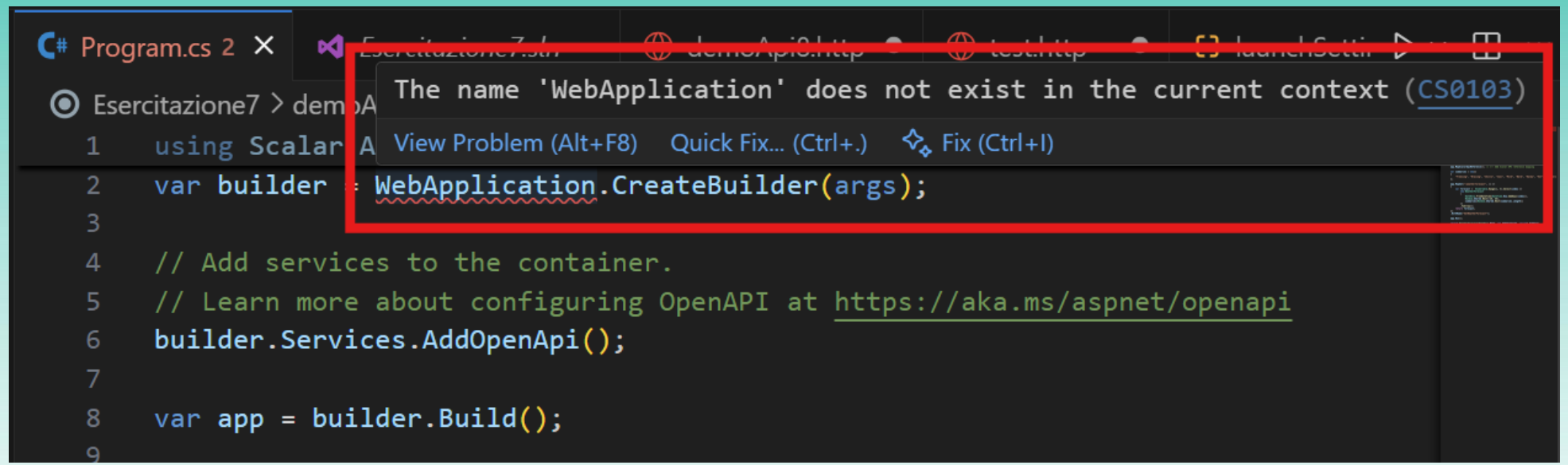


Nel pannello Debug, selezionando il menu sarà disponibile la nuova voce ‘Launch Both APIs’ come l’avevamo chiamata nel file di configurazione.

Selezionare e lanciare da F5 o da pulsante Play accanto

COESIONE ITALIA 21-27 MARCHE
Cofinanziato dall'Unione europea
REGIONE MARCHE
its TURISMO MARCHE
TURISMO E NUOVE TECNOLOGIE

Risoluzione launch progetti multipli ed errori



Per la risoluzione si rimanda alla slide successiva >>